

Phase 1 — Completion Report

Project: Virtualfactor IT CMDB

Repository: <https://github.com/jallamasc/vf-cmdb>

Branch: `feature/phase-1-quick-add-rack-hierarchy` (PR #3 → `master`)

Date completed: 2026-07-23

User: Alejandro (jallamasc), Virtualfactor, Bogotá, Colombia

1. Scope Delivered

Phase 1 grew beyond the original preparation doc. In addition to the Quick-Add UI and rack hierarchy, the user requested a full naming-convention engine and several deployment-hardening fixes. All of the following shipped:

1.1 Physical hierarchy (Site → Datacenter → Floor → Room → Rack)

- **New tables:** `datacenters` , `datacenter_floors` , `rooms` , `rack_types` .
- `racks` extended with `datacenter_floor_id` , `room_id` , `rack_type_id` , `code` , and `total_units` .
- `rack_types` reference entity manages standard rack heights/categories (16U, 32U, 42U, 48U seeded) instead of a free-form height field.
- Alembic migration `0003_hierarchy_and_naming.py` — idempotent/guarded (helper predicates `_has_table` , `_has_column` , `_has_fk` , `_has_check` , `_has_index`).

1.2 Quick-Add UI

- `frontend/src/pages/Hierarchy.tsx` — inline **collapsible** forms (not modals) at the top of the page for every hierarchy level, per the agreed UI pattern.
- Cascading parent selectors (Site → Datacenter → Floor → Room → Rack).
- Wired into routing (`App.tsx`) and navigation (`Layout.tsx` , “Physical Hierarchy” under “Sites & Physical”).

1.3 Naming-convention engine

Implemented in `backend/app/abbrev.py` , enforced in `backend/app/crud.py` , exposed via `backend/app/routers/special.py` , and surfaced in the UI via `frontend/src/components/AbbrevField.tsx` .

Feature	Rule delivered
Case enforcement	Per record-type setting <code>case_enforcement</code> $\in$ { uppercase , lowercase , mixed }. Applied to code/abbreviation and relevant name fields.
Allowed characters	Domain-name charset [A-Za-z0-9-] , no leading/trailing hyphen, no consecutive -- . Enforced as DB CHECK + server validation + client DOMAIN_RE .
Trailing sequence number	<code>name_prefix</code> + <code>sequence_number</code> columns on 5 device tables. No zero-padding.
Gap reuse	GET /naming/gaps?prefix=... returns used numbers, gaps, next sequential, and a recommendation. UI (SequenceGapHelper.tsx) prompts the user to reuse a gap or continue sequentially.
Trim modes	<code>trim_mode</code> $\in$ { manual , first_1 , first_2 , first_3 , first_4 , acronym , consonants }. Live preview via GET /naming/preview .
Global uniqueness	abbreviation_registry table with a functional unique index uq_abbreviation_registry_lower on lower(abbreviation) — case-insensitive uniqueness across all entity types. GET /naming/check-abbreviation powers the live check.

1.4 Seed corrections

- Site “Home” → code `hm` (not `hq` ; `hq` reserved for a future “Head Quarters”).
- Resolved **10** pre-existing abbreviation collisions / charset violations in `seed.py` (e.g. “Virtual Server” `vs` → `vse`), and every lookup + hierarchy item is now registered in `abbreviation_registry` via `abbrev.sync_registry` .

1.5 Deployment hardening (added this phase — see incident below)

- `deploy-podman.sh` : idempotent `up / restart` , new `reset` and `recover` subcommands.
 - `podman-compose.yml` : **pinned** volume names (`vf-cmdb_pgdata` , `vf-cmdb_pgadmin`), overridable via `${PGDATA_VOLUME}` / `${PGADMIN_VOLUME}` .
 - Quadlet volume units unified to the **underscore** names to match compose.
 - `deploy/scripts/vf-cmdb-backup.sh` : pgAdmin volume name corrected to underscore form.
 - New `deploy/scripts/recover-podman-volumes.sh` (4-phase, data-safe recovery).
-

2. Verification Results

- **Backend:** `alembic upgrade head` and `python -m app.seed` ran successfully. Smoke tests (httpx) confirmed naming preview, gap detection, and conflict rejection (422/409) behave as designed.
- **Frontend:** `npm run build` (`tsc -b && vite build`) compiles cleanly.
- **End-to-end:** After the deployment incident was resolved (below), the user confirmed the app is reachable and functional again.

3. Deployment Incident & Resolution (Podman volume-store corruption)

Symptom

On repeated `podman-compose up`, the stack failed with:

```
Error: creating container storage: the container name "vf_cmdb_db" is already in use ...
Error: inspecting volume vf-cmdb_pgadmin: more than one result for volume name ... :
volume already exists
... later spreading to vf-cmdb_pgdata as well.
```

Root cause

Two deployment paths had been run on the same host:

- **podman-compose** → created underscore-named volumes (`vf-cmdb_pgadmin`).
- **Quadlet/systemd** → created dash-named volumes (`vf-cmdb-pgadmin`, `vf-cmdb-pgdata`).

The overlapping create/inspect operations left **duplicate name keys** in podman's internal volume database (boltdb). `podman volume inspect <name>` then returned "more than one result for volume name", which `podman-compose` cannot recover from, and each aborted run left half-created containers whose names were still reserved (the cascading "container name already in use" errors).

What was tried (in order)

1. Idempotent container cleanup + targeted `podman volume rm` of the pgadmin variants — cleared pgadmin but not the corrupted key.
2. `podman system renumber` (rebuilds ID/lock DB, non-destructive) — **did not** dedupe the duplicate volume-name key in podman 4.9.3.
3. A non-destructive "fresh-volume switch" script path (create new volume, copy data, repoint via `PGDATA_VOLUME`).

Final resolution

The user ran `podman system reset -f`, which cleared the corrupted store entirely. Images rebuilt from the Containerfiles and the stack came back up with access restored.

Note on data: a `podman system reset -f` wipes all volumes. In this case the reset resolved access and the environment is essentially a fresh data plane (seeded on boot). The `recover-podman-volumes.sh` script now performs an on-disk `pg_dump` -independent **file backup before** any destructive step, so a future occurrence can be repaired without data loss.

Preventative measures now in the repo

- **Single source of volume names** across compose and Quadlet (underscore form).
- `./deploy-podman.sh recover` — Phase 1 backs up on-disk PostgreSQL files, Phase 2 attempts least-destructive repair, Phase 3 switches to a fresh volume name (non-destructive), Phase 4 (guarded by `CONFIRM_RESET=1`) does the full reset and restores from the Phase-1 backup.
- **Operational rule:** use **only one** deployment path per host — either `deploy-podman.sh up` (compose) **or** the Quadlet/systemd units, never both.

4. Lessons Learned / Carry-Forward Rules

1. **Never mix compose and Quadlet on the same host** — pick one lifecycle manager.
2. **Pin volume names explicitly** so no tool auto-prefixes divergent names.
3. **Back up on-disk volume files before any** `podman system reset` — data lives as plain files under the volume mountpoint, independent of the (corruptible) metadata DB.
4. Naming-convention enforcement belongs in the **application/CRUD layer** (clean 422/409) and the **DB layer** (CHECK/unique) — belt and suspenders.

5. Commit Trail (branch `feature/phase-1-quick-add-rack-hierarchy`)

Commit	Summary
<code>fce6f08</code>	Backend: hierarchy tables + global naming registry + migration + seed
<code>f9999ad</code>	Frontend: Quick Add hierarchy + abbreviation field + gap prompt
<code>90eec80</code>	deploy: idempotent up/restart + <code>reset</code> for stale podman state
<code>696e5b1</code>	deploy: pin volume names + robust reset
<code>5443aac</code>	deploy: add <code>recover-podman-volumes.sh</code>
<code>89f1d7e</code>	deploy: non-destructive fresh-volume recovery

6. Status

✅ **Phase 1 complete and merged-ready (PR #3).** App verified running by the user.
Ready to proceed to **Phase 2** — see `PHASE_2_PREPARATION.md`.